Implementação e Análise de Estratégias de Algoritmos para Solução de Problema NP

Ciclo Hamiltoniano Direcionado

Autores

Nome completo	Matrícula
Fabio Firanzi	[**matrícula**]
Finéias Antonio	[**matrícula**]
Heitor Vaz	[**matrícula**]
Tulio Vitorette	[**matrícula**]

Sumário

1 Introdução **3**

1.1 O problema . . . 3

1.2 Dois exemplos mínimos . . . 3

1.3 Aplicações práticas . . . 4

2 Prova: CHD pertence à classe NP **5**

2.1 CHD $\in$ NP . . . 5

2.2 CHD é NP-completo (além do que o edital pede) . . . 5

3 Implementação **6**

3.1 Base comum . . . 6

3.2 Força bruta . . . 13

3.3 Backtracking . . . 14

3.4 Divisão e conquista: por que não há algoritmo . . . 17

3.5 Programação dinâmica (Held-Karp) . . . 19

3.6 Guloso . . . 21

3.7 Heurística: recozimento simulado . . . 24

3.8 Caso especial: torneios (Camion) . . . 28

4 Análise de complexidade **31**
4.1 Tempo e espaço teóricos 31
4.2 O que o contador mede 32

5 Análise de resultados **33**
5.1 Método 33
5.2 Os três exatos no pior caso 34
5.3 Efeito do filtro de conectividade forte 35
5.4 Variação entre famílias 35
5.5 Guloso e recozimento: quem acha o ciclo? 36
5.6 Camion em torneios 37

6 Conclusão **37**

1 Introdução

1.1 O problema

Seja $G = (V, A)$ um grafo direcionado simples, sem laços, com $n = |V| \geq 2$ vértices. Um *ciclo hamiltoniano direcionado* é uma sequência $(v_1, v_2, \ldots, v_n)$ tal que

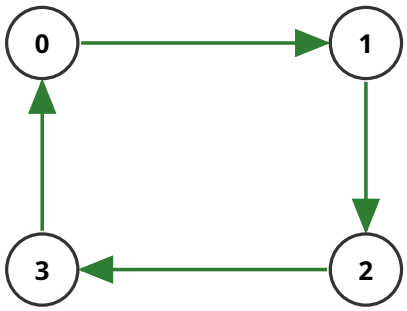
- os v_i são todos distintos e cobrem V;
- $(v_i, v_{i+1}) \in A$ para $i = 1, \ldots, n-1$;
- $(v_n, v_1) \in A$.

O **problema de decisão (CHD)** pergunta: dado G, existe um ciclo hamiltoniano direcionado? O código deste trabalho resolve a *versão de busca*: devolve a sequência quando ela existe. A resposta de decisão é o sim ou o não dessa busca, e a própria sequência é a prova de que a resposta é sim, usada na Seção 2.

Convenção de simetria. Rotacionar um ciclo não cria outro ciclo. Por isso todos os métodos exatos fixam $v_1 = 0$. Inverter o sentido, ao contrário, produz outro ciclo direcionado e não é equivalente: o sentido oposto continua valendo e entra na busca.

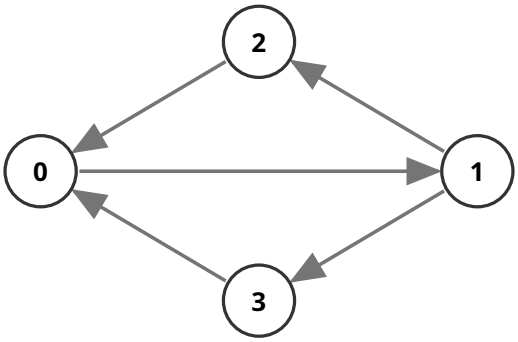
1.2 Dois exemplos mínimos

Exemplo com ciclo. Arcos $(0,1), (1,2), (2,3), (3,0)$. O ciclo é $0 \to 1 \to 2 \to 3 \to 0$, em verde na figura.



Digrafo com ciclo hamiltoniano direcionado.

Exemplo sem ciclo, com o digrafo forte. Arcos $(0,1), (1,2), (2,0), (1,3), (3,0)$. De 3 só se sai para 0, e em 3 só se entra por 1, então 3 teria de ocupar o trecho $1 \to 3 \to 0$. O vértice 2 não cabe nesse trecho com os arcos existentes. O digrafo é fortemente conexo: conectividade forte é condição *necessária* e não suficiente.



Digrafo fortemente conexo sem ciclo hamiltoniano.

Os dois exemplos podem ser executados por qualquer programa do trabalho, com as famílias `ciclo_simples` e `forte_sem_ciclo`.

1.3 Aplicações práticas

Em cada aplicação abaixo, o que é vértice, o que é arco e por que a volta ao início importa. A dificuldade é a mesma nos quatro casos: o ciclo hamiltoniano é NP-completo, e a solução exata só funciona para digrafos pequenos.

Entregas urbanas (Correios, iFood, Amazon). Cada endereço de entrega é um vértice, e um arco é o trecho de rua que leva de um endereço a outro. Como há ruas de mão única, ir de A a B não equivale a ir de B a A, e o digrafo é naturalmente *direcionado*. O entregador parte do depósito, visita cada endereço uma vez e volta ao depósito: a volta ao início é o retorno da van ou da moto à base. As empresas resolvem a versão com custos (caixeiro viajante assimétrico e roteirização de veículos), e o CHD é a pergunta de viabilidade dentro dela: o caso em que todo arco existente custa 1 e o limite é $B = n$. Com centenas de paradas por dia, a solução exata é inviável (Seção 4) e as empresas usam heurísticas, como o recozimento deste trabalho. O texto não atribui a elas razão de aproximação provada.

Computação com DNA (Adleman, 1994). A primeira demonstração de computação com moléculas de DNA resolveu um problema de caminho hamiltoniano num *digrafo* de 7 vértices [1]. Cada cidade foi codificada por uma sequência de DNA e cada arco por uma sequência que liga o fim de uma cidade ao início da seguinte. Misturadas num tubo de ensaio, as moléculas formaram, em paralelo, todos os caminhos possíveis, e as etapas seguintes eliminaram os que não passavam por todas as cidades exatamente uma vez. O experimento é famoso por mostrar a ideia, e também por mostrar o limite: o número de moléculas necessárias cresce com o número de caminhos, isto é, de forma exponencial, e a abordagem não escala.

Montagem de genomas (Projeto Genoma Humano e bactérias). Para sequenciar um genoma, a máquina lê milhões de fragmentos curtos. Na formulação clássica, cada fragmento é um vértice, e um arco liga dois fragmentos quando o final de um se sobrepõe ao começo do outro. Uma sequência que usa cada fragmento uma vez é um

caminho hamiltoniano; no genoma circular de uma bactéria ou de um plasmídeo é um ciclo hamiltoniano, e a volta ao início é a própria circularidade do DNA. Como o problema é NP-completo, os montadores modernos mudaram de modelo: usam o grafo de de Bruijn e o caminho euleriano, que é polinomial [5]. O problema deste trabalho explica uma mudança real de abordagem numa área muito conhecida.

Campeonato por pontos corridos (Brasileirão, xadrez). Num campeonato em que todos jogam contra todos, os competidores são vértices e uma vitória é um arco do vencedor para o perdedor. Todo par tem exatamente um sentido (sem empate, ou desconsiderando-o), então o digrafo é um torneio. Se ele é fortemente conexo, o teorema de Camion [4] garante um ciclo hamiltoniano: uma ordenação circular em que cada competidor venceu o seguinte, e o último venceu o primeiro. O ciclo explica por que o "ganhou de" não é transitivo (A vence B, B vence C, C vence A). Este é o caso em que a pergunta deixa de ser NP-completa: a construção da Seção 3.8 acha o ciclo em tempo polinomial, e mostra que a dificuldade mora na estrutura do digrafo e não no nome do problema.

2 Prova: CHD pertence à classe NP

2.1 CHD $\in$ NP

Um problema está em NP quando, se alguém propõe uma resposta "sim", dá para conferi-la rapidamente, em tempo polinomial [6, 12]. Não é preciso saber *achar* a resposta, só *conferir*.

No CHD, a resposta "sim" proposta é uma sequência de vértices, $c = (c_1, \ldots, c_n)$. Para conferir se ela é um ciclo hamiltoniano direcionado (função `verifica_ciclo`, Listagem 5), basta checar três coisas:

1. A sequência tem exatamente n vértices.
2. Nenhum vértice se repete. Um vetor de n marcas resolve em n passos. Como são n vértices diferentes em n posições, nenhum vértice fica de fora.
3. Cada vértice tem arco para o próximo, e o último tem arco para o primeiro. São n consultas à matriz, cada uma em tempo constante.

Ao todo são cerca de n passos, um tempo polinomial. A conferência aceita se, e somente se, c é de fato um ciclo hamiltoniano direcionado de G. Logo, CHD está em NP.

2.2 CHD é NP-completo (além do que o edital pede)

Um problema é NP-completo quando está em NP e é pelo menos tão difícil quanto os problemas mais difíceis de NP. Para mostrar isso, usa-se um problema que já se sabe ser difícil: o ciclo hamiltoniano em grafos *sem direção*, que é NP-completo [10, 8].

A ideia é transformar um problema no outro:

1. Pegue um grafo sem direção e troque cada aresta entre u e v por dois arcos, um de u para v e outro de v para u. Isso gera um digrafo.

2. Se o grafo tem um ciclo hamiltoniano, o digrafo tem o mesmo ciclo, pois todo caminho existe nos dois sentidos.

3. Se o digrafo tem um ciclo hamiltoniano, o grafo também tem, pois cada arco usado veio de uma aresta.

Logo, quem soubesse resolver o CHD saberia resolver o ciclo hamiltoniano sem direção, que é um problema difícil. Então o CHD também é difícil e, como já está em NP, é NP-completo (problema GT38 de [8]).

3 Implementação

3.1 Base comum

- `Digrafo`: matriz de adjacência (consulta de arco em $O(1)$) e listas de saída e de entrada ordenadas (Listagem 3, 4). A conectividade forte usa o algoritmo de Kosaraju, em $O(n^2)$ com listas de adjacência de tamanho $O(n^2)$.

- `Contador`: contagem de instruções, separada do tempo de parede. Três campos: `consultas_aresta` (leituras na matriz), `escritas` (extensão do caminho ou preenchimento de um estado da PD) e `nos` (nós da árvore, estados da PD ou movimentos da heurística, conforme o algoritmo). Os três não são somados entre estratégias: uma transição de Held-Karp e um teste de permutação não são a mesma operação.

- `Relogio`: tempo de parede com `std::chrono::steady_clock`. O estouro do prazo (30 s) lança `TempoEsgotado` e é registrado como dado censurado, nunca como "o ciclo não existe".

- `verifica_ciclo`: o verificador da prova.

- `executa`: cada estratégia tem o seu próprio programa (um `main.cpp` na pasta da estratégia). Todos usam esta função para ler a família e o tamanho n da linha de comando, gerar o digrafo, medir o tempo, conferir o ciclo com `verifica_ciclo` e imprimir uma linha com o resultado, os três contadores e o tempo.

Listing 1: `contador.hpp`

```
#pragma once

namespace chd {

// Contagem separada do tempo de parede. Os três campos não são somados
// entre estratégias: uma transição de Held-Karp e um teste de permutação
// não são a mesma operação.
struct Contador {
  long long consultas_aresta = 0;
  long long escritas = 0;
  long long nos = 0;

  void zera() {
    consultas_aresta = 0;
    escritas = 0;
```

```
    nos = 0;
  }
};

}  // namespace chd
```

Listing 2: `tempo.hpp`

```
#pragma once

#include <chrono>
#include <optional>
#include <stdexcept>

namespace chd {

// Estouro de prazo não é resposta "inexistente". Quem chama distingue os dois.
class TempoEsgotado : public std::runtime_error {
 public:
  TempoEsgotado() : std::runtime_error("tempo esgotado") {}
};

struct Relogio {
  std::optional<std::chrono::steady_clock::time_point> limite;

  void verifica() const {
    if (limite && std::chrono::steady_clock::now() >= *limite) {
      throw TempoEsgotado();
    }
  }
};

}  // namespace chd
```

Listing 3: `digrafo.hpp`

```
#pragma once

#include "contador.hpp"

#include <vector>

namespace chd {

// Digrafo simples, vértices 0..n-1, sem laço. Matriz para consulta O(1)
// e listas de saída e de entrada para a busca.
class Digrafo {
 public:
  explicit Digrafo(int n);

  int n() const { return n_; }

  void adiciona_arco(int u, int v);

  // Leitura da matriz sem contar. Geradores e checagens internas usam esta.
  bool tem_arco(int u, int v) const;

  // Cada chamada conta uma consulta de aresta, mesmo quando o arco falta.
```

```
  bool consulta_arco(int u, int v, Contador& contador) const;

  const std::vector<int>& saida(int u) const { return saida_[u]; }
  const std::vector<int>& entrada(int u) const { return entrada_[u]; }

  // Condição necessária para um ciclo hamiltoniano, e não suficiente.
  bool fortemente_conexo() const;

  // Exatamente um sentido em cada par, sem laço.
  bool eh_torneio() const;

 private:
  int n_;
  std::vector<std::vector<char>> matriz_;
  std::vector<std::vector<int>> saida_;
  std::vector<std::vector<int>> entrada_;
};

}  // namespace chd
```

Listing 4: `digrafo.cpp`

```
#include "00-comum/digrafo.hpp"

#include <algorithm>
#include <stdexcept>

namespace chd {

Digrafo::Digrafo(int n)
    : n_(n),
      matriz_(n, std::vector<char>(n, 0)),
      saida_(n),
      entrada_(n) {
  if (n < 0) {
    throw std::invalid_argument("digrafo com ordem negativa");
  }
}

void Digrafo::adiciona_arco(int u, int v) {
  if (u < 0 || v < 0 || u >= n_ || v >= n_) {
    throw std::out_of_range("vertice fora do digrafo");
  }
  if (u == v || matriz_[u][v]) {
    return;
  }
  matriz_[u][v] = 1;
  auto insere = [](std::vector<int>& lista, int x) {
    auto it = std::lower_bound(lista.begin(), lista.end(), x);
    lista.insert(it, x);
  };
  insere(saida_[u], v);
  insere(entrada_[v], u);
}

bool Digrafo::tem_arco(int u, int v) const {
  return matriz_[u][v] != 0;
}
```

```cpp

bool Digrafo::consulta_arco(int u, int v, Contador& contador) const {
  contador.consultas_aresta++;
  return matriz_[u][v] != 0;
}

bool Digrafo::fortemente_conexo() const {
  if (n_ == 0) {
    return true;
  }
  std::vector<char> visto(n_, 0);
  std::vector<int> ordem;
  ordem.reserve(n_);

  auto dfs_saida = [&](auto&& self, int u) -> void {
    visto[u] = 1;
    for (int v : saida_[u]) {
      if (!visto[v]) {
        self(self, v);
      }
    }
    ordem.push_back(u);
  };
  for (int u = 0; u < n_; ++u) {
    if (!visto[u]) {
      dfs_saida(dfs_saida, u);
    }
  }

  std::vector<char> visto_volta(n_, 0);
  int componentes = 0;
  auto dfs_entrada = [&](auto&& self, int u) -> void {
    visto_volta[u] = 1;
    for (int v : entrada_[u]) {
      if (!visto_volta[v]) {
        self(self, v);
      }
    }
  };
  for (int i = n_ - 1; i >= 0; --i) {
    int u = ordem[i];
    if (!visto_volta[u]) {
      dfs_entrada(dfs_entrada, u);
      componentes++;
    }
  }
  return componentes == 1;
}

bool Digrafo::eh_torneio() const {
  for (int u = 0; u < n_; ++u) {
    if (matriz_[u][u]) {
      return false;
    }
    for (int v = u + 1; v < n_; ++v) {
      if (matriz_[u][v] == matriz_[v][u]) {
        return false;
      }
```

```
    }
  }
  return true;
}

}  // namespace chd
```

Listing 5: `verifica.cpp`

```
#include "00-comum/verifica.hpp"

namespace chd {

bool verifica_ciclo(const Digrafo& g, const std::vector<int>& ciclo, Contador&
    contador) {
  const int n = g.n();
  if (static_cast<int>(ciclo.size()) != n || n < 2) {
    return false;
  }
  std::vector<char> visto(n, 0);
  for (int v : ciclo) {
    if (v < 0 || v >= n || visto[v]) {
      return false;
    }
    visto[v] = 1;
  }
  for (int i = 0; i < n; ++i) {
    int proximo = ciclo[(i + 1) % n];
    if (!g.consulta_arco(ciclo[i], proximo, contador)) {
      return false;
    }
  }
  return true;
}

}  // namespace chd
```

Listing 6: `execucao.hpp`

```
#pragma once

#include "00-comum/digrafo.hpp"
#include "00-comum/tempo.hpp"

#include <functional>
#include <map>
#include <optional>
#include <string>
#include <vector>

namespace chd {

struct Contexto {
  int n = 0;
  int rep = 0;
  std::map<std::string, std::string> opcoes;

  std::string opcao(const std::string& nome, const std::string& padrao) const;
```

```
};

struct Resultado {
  std::optional<std::vector<int>> ciclo;
  std::string motivo;
  int fitness = -1;
};

using Solucionador = std::function<Resultado(const Digrafo&, const Contexto&, Contador
    &, const Relogio&)>;

int executa(int argc, char** argv, const std::string& algoritmo, const Solucionador&
    resolve);

}  // namespace chd
```

Listing 7: execucao.cpp

```
#include "00-comum/execucao.hpp"

#include "00-comum/gerador.hpp"
#include "00-comum/verifica.hpp"

#include <chrono>
#include <exception>
#include <iostream>
#include <random>

namespace chd {

std::string Contexto::opcao(const std::string& nome, const std::string& padrao) const
    {
  auto it = opcoes.find(nome);
  return it == opcoes.end() ? padrao : it->second;
}

int executa(int argc, char** argv, const std::string& algoritmo, const Solucionador&
    resolve) {
  std::vector<std::string> posicionais;
  Contexto base;
  for (int i = 1; i < argc; ++i) {
    const std::string arg = argv[i];
    if (arg.rfind("--", 0) == 0) {
      const size_t igual = arg.find('=');
      if (igual == std::string::npos) {
        base.opcoes[arg.substr(2)] = "1";
      } else {
        base.opcoes[arg.substr(2, igual - 2)] = arg.substr(igual + 1);
      }
    } else {
      posicionais.push_back(arg);
    }
  }
  if (posicionais.size() < 2) {
    std::cerr << "uso: " << argv[0] << " <familia> <n> [repeticoes] [--opcao=valor]\n"
    ;
    return 2;
  }
```

```
38  const std::optional<Familia> familia = familia_por_nome(posicionais[0]);
39  if (!familia) {
40    std::cerr << "familia␣desconhecida:␣" << posicionais[0] << "\n";
41    return 2;
42  }
43  const int n = std::stoi(posicionais[1]);
44  const int repeticoes = posicionais.size() > 2 ? std::stoi(posicionais[2]) : 1;
45
46  for (int rep = 0; rep < repeticoes; ++rep) {
47    std::mt19937 rng(20261107u + static_cast<unsigned>(*familia) * 100000u +
      static_cast<unsigned>(n) * 1000u +
48                     static_cast<unsigned>(rep));
49    const Digrafo g = gera(*familia, n, rng);
50
51    Contexto contexto = base;
52    contexto.n = g.n();
53    contexto.rep = rep;
54    Contador contador;
55    Relogio relogio;
56    relogio.limite = std::chrono::steady_clock::now() + std::chrono::seconds(30);
57    Resultado resultado;
58    std::string status;
59    std::string erro;
60
61    const auto inicio = std::chrono::steady_clock::now();
62    try {
63      resultado = resolve(g, contexto, contador, relogio);
64      status = resultado.motivo;
65    } catch (const TempoEsgotado&) {
66      status = "censurado";
67    } catch (const std::exception& e) {
68      status = "recusado";
69      erro = e.what();
70    }
71    const double tempo_ms =
72        std::chrono::duration<double, std::milli>(std::chrono::steady_clock::now() -
      inicio).count();
73
74    if (resultado.ciclo) {
75      Contador extra;
76      status = verifica_ciclo(g, *resultado.ciclo, extra) ? "ciclo" : "invalido";
77    }
78
79    std::cout << algoritmo << "␣" << posicionais[0] << "␣n=" << g.n() << "␣rep=" <<
      rep << "␣status=" << status
80              << "␣nos=" << contador.nos << "␣consultas=" << contador.consultas_aresta
81              << "␣escritas=" << contador.escritas << "␣tempo_ms=" << tempo_ms;
82    if (resultado.fitness >= 0) {
83      std::cout << "␣fitness=" << resultado.fitness;
84    }
85    if (!erro.empty()) {
86      std::cout << "␣motivo=\"" << erro << "\"";
87    }
88    if (resultado.ciclo && g.n() <= 30) {
89      std::cout << "␣ciclo=";
90      for (size_t i = 0; i < resultado.ciclo->size(); ++i) {
91        std::cout << (i ? "," : "") << (*resultado.ciclo)[i];
92      }
```

```
    }
    std::cout << "\n";
  }
  return 0;
}

}  // namespace chd
```

3.2 Força bruta

Fixa o vértice 0 na origem e gera as $(n-1)!$ permutações dos demais com `std::next_permutation`. Para cada permutação consulta os n arcos do ciclo e para na primeira que fecha. Não há poda: poda é o que distingue o backtracking na comparação.

Listing 8: `forca_bruta.cpp`

```
#include "01-forca-bruta/forca_bruta.hpp"

#include <algorithm>

namespace chd {

std::optional<std::vector<int>> forca_bruta(const Digrafo& g, Contador& contador,
                                            const Relogio& relogio) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }
  // Filtro necessário. Não é poda da árvore de permutações.
  if (!g.fortemente_conexo()) {
    return std::nullopt;
  }

  std::vector<int> perm(n - 1);
  for (int i = 0; i < n - 1; ++i) {
    perm[i] = i + 1;
  }

  std::vector<int> ciclo(n);
  long long passo = 0;
  do {
    if ((passo++ & 4095) == 0) {
      relogio.verifica();
    }
    contador.nos++;
    ciclo[0] = 0;
    for (int i = 0; i < n - 1; ++i) {
      ciclo[i + 1] = perm[i];
    }
    contador.escritas += n;

    bool fecha = true;
    for (int i = 0; i < n; ++i) {
      int proximo = ciclo[(i + 1) % n];
      if (!g.consulta_arco(ciclo[i], proximo, contador)) {
        fecha = false;
        break;
```

```
      }
    }
    if (fecha) {
      return ciclo;
    }
  } while (std::next_permutation(perm.begin(), perm.end()));

  return std::nullopt;
}

}  // namespace chd
```

Listing 9: `01-forca-bruta/main.cpp`

```
#include "00-comum/execucao.hpp"
#include "01-forca-bruta/forca_bruta.hpp"

int main(int argc, char** argv) {
  return chd::executa(argc, argv, "forca_bruta",
                      [](const chd::Digrafo& g, const chd::Contexto&, chd::Contador&
    contador,
                         const chd::Relogio& relogio) {
                        chd::Resultado resultado;
                        resultado.ciclo = chd::forca_bruta(g, contador, relogio);
                        resultado.motivo = "inexistente";
                        return resultado;
                      });
}
```

3.3 Backtracking

Busca em profundidade a partir de 0. O próximo vértice é um vizinho de saída ainda fora do caminho; com n vértices no caminho, o único teste é o arco de volta a 0. Duas ordens de vizinhos (mesma resposta, árvores de tamanhos diferentes): índice crescente e a regra de Warnsdorff adaptada (primeiro o vizinho com menos saídas livres, ideia herdada do passeio do cavalo, sem herdar uma prova). A ordenação é detalhe do backtracking, não um algoritmo guloso. O parâmetro `usar_filtro` permite a ablação da poda por conectividade forte.

Listing 10: `backtracking.cpp`

```
#include "02-backtracking/backtracking.hpp"

#include <algorithm>
#include <utility>

namespace chd {
namespace {

struct Item {
  int vertice;
  int chave;
};

}  // namespace

```

```cpp
std::optional<std::vector<int>> backtracking(const Digrafo& g, Contador& contador,
                                             OrdemBusca ordem, bool usar_filtro,
                                             const Relogio& relogio) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }
  if (usar_filtro && !g.fortemente_conexo()) {
    return std::nullopt;
  }

  std::vector<int> caminho;
  caminho.reserve(n);
  caminho.push_back(0);
  std::vector<char> usado(n, 0);
  usado[0] = 1;
  long long passo = 0;

  auto saidas_livres = [&](int v) {
    int livres = 0;
    for (int w : g.saida(v)) {
      contador.consultas_aresta++;
      if (!usado[w]) {
        livres++;
      }
    }
    return livres;
  };

  auto dfs = [&](auto&& self) -> bool {
    if ((passo++ & 1023) == 0) {
      relogio.verifica();
    }
    contador.nos++;
    if (static_cast<int>(caminho.size()) == n) {
      // Os n vértices já estão no caminho. Só resta o arco de volta à origem.
      return g.consulta_arco(caminho.back(), 0, contador);
    }

    const int u = caminho.back();
    std::vector<Item> candidatos;
    candidatos.reserve(g.saida(u).size());
    for (int v : g.saida(u)) {
      if (usado[v]) {
        continue;
      }
      int chave = 0;
      if (ordem == OrdemBusca::Warnsdorff) {
        chave = saidas_livres(v);
      }
      candidatos.push_back({v, chave});
    }
    if (ordem == OrdemBusca::Warnsdorff) {
      std::sort(candidatos.begin(), candidatos.end(), [](const Item& a, const Item& b)
     {
        if (a.chave != b.chave) {
          return a.chave < b.chave;
        }
```

```
        return a.vertice < b.vertice;
      });
    } else {
      std::sort(candidatos.begin(), candidatos.end(), [](const Item& a, const Item& b)
     {
        return a.vertice < b.vertice;
      });
    }

    for (const Item& item : candidatos) {
      usado[item.vertice] = 1;
      caminho.push_back(item.vertice);
      contador.escritas++;
      if (self(self)) {
        return true;
      }
      caminho.pop_back();
      usado[item.vertice] = 0;
    }
    return false;
  };

  if (dfs(dfs)) {
    return caminho;
  }
  return std::nullopt;
}

}  // namespace chd
```

Listing 11: 02-backtracking/main.cpp

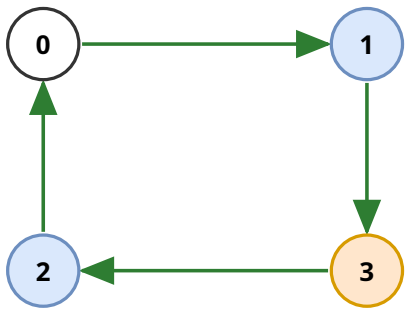
```
#include "00-comum/execucao.hpp"
#include "02-backtracking/backtracking.hpp"

int main(int argc, char** argv) {
  return chd::executa(argc, argv, "backtracking",
                      [](const chd::Digrafo& g, const chd::Contexto& contexto, chd::
    Contador& contador,
                         const chd::Relogio& relogio) {
                       const bool indice = contexto.opcao("ordem", "warnsdorff") == "
    indice";
                       const bool filtro = contexto.opcao("sem-filtro", "0") != "1";
                       chd::Resultado resultado;
                       resultado.ciclo = chd::backtracking(
                           g, contador, indice ? chd::OrdemBusca::Indice : chd::
    OrdemBusca::Warnsdorff, filtro,
                           relogio);
                       resultado.motivo = "inexistente";
                       return resultado;
                     });
}
```

3.4 Divisão e conquista: por que não há algoritmo

Divisão e conquista, no sentido de [6], parte a entrada em subproblemas *independentes*, resolve cada um e combina as respostas. No CHD a combinação não é independente: um ciclo hamiltoniano de G não é a união de ciclos hamiltonianos dos subdigrafos induzidos por uma partição $V = L \cup R$, porque o ciclo pode alternar entre L e R várias vezes.

O atalho que *parece* divisão (permutar um bloco L, permutar o outro bloco R e concatenar) responde outra pergunta. Contraexemplo, com origem 0, $L = \{1, 2\}$ e $R = \{3\}$: as concatenações possíveis são só $(0, 1, 2, 3)$, $(0, 2, 1, 3)$, $(0, 3, 1, 2)$ e $(0, 3, 2, 1)$. O digrafo de arcos $(0, 1), (1, 3), (3, 2), (2, 0)$ tem o ciclo $0 \to 1 \to 3 \to 2 \to 0$ e nenhuma das quatro sequências, então o método de blocos responde *não* num digrafo que *tem* ciclo.



O ciclo $0 \to 1 \to 3 \to 2 \to 0$ intercala os blocos $L = \{1, 2\}$ (azul) e $R = \{3\}$ (laranja), e nenhuma concatenação de blocos o produz.

Por isso o grupo **não apresenta** esse atalho como solução. A Listagem 12 existe apenas para demonstrar o falso negativo (o programa `ciclo_por_blocos`, com a família `ciclo_intercalado`, o exibe) e fica limitada a $n \leq 10$.

A decomposição que funciona usa outro subproblema: "caminho simples que começa em 0, cobre o conjunto S e termina em v". Esses subproblemas se repetem ao longo da busca; resolver cada um uma vez e reutilizar é programação dinâmica, a próxima subseção. Algoritmos mais recentes (por exemplo, a inclusão-exclusão de Björklund para o caso não direcionado [3]) ficam fora do código, e o algoritmo exato de referência do trabalho é Held-Karp.

Listing 12: `ciclo_por_blocos.cpp`

```cpp
#include "03-divisao-e-conquista/ciclo_por_blocos.hpp"

#include <algorithm>
#include <stdexcept>

namespace chd {

std::optional<std::vector<int>> ciclo_por_blocos_contiguos(const Digrafo& g, Contador&
    contador) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }
  if (n > 10) {
```

```
    throw std::invalid_argument("ciclo␣por␣blocos␣so␣enumera␣ate␣n␣=␣10");
  }

  std::vector<int> resto;
  resto.reserve(n - 1);
  for (int v = 1; v < n; ++v) {
    resto.push_back(v);
  }
  const int meio = (n - 1 + 1) / 2;
  std::vector<int> esquerda(resto.begin(), resto.begin() + meio);
  std::vector<int> direita(resto.begin() + meio, resto.end());
  std::sort(esquerda.begin(), esquerda.end());

  auto fecha = [&](const std::vector<int>& ordem) {
    contador.nos++;
    contador.escritas += n;
    for (int i = 0; i < n; ++i) {
      if (!g.consulta_arco(ordem[i], ordem[(i + 1) % n], contador)) {
        return false;
      }
    }
    return true;
  };

  do {
    std::vector<int> bloco_direito = direita;
    std::sort(bloco_direito.begin(), bloco_direito.end());
    do {
      std::vector<int> esquerda_depois;
      esquerda_depois.reserve(n);
      esquerda_depois.push_back(0);
      esquerda_depois.insert(esquerda_depois.end(), esquerda.begin(), esquerda.end());
      esquerda_depois.insert(esquerda_depois.end(), bloco_direito.begin(),
    bloco_direito.end());
      if (fecha(esquerda_depois)) {
        return esquerda_depois;
      }

      std::vector<int> direita_depois;
      direita_depois.reserve(n);
      direita_depois.push_back(0);
      direita_depois.insert(direita_depois.end(), bloco_direito.begin(), bloco_direito
    .end());
      direita_depois.insert(direita_depois.end(), esquerda.begin(), esquerda.end());
      if (fecha(direita_depois)) {
        return direita_depois;
      }
    } while (std::next_permutation(bloco_direito.begin(), bloco_direito.end()));
  } while (std::next_permutation(esquerda.begin(), esquerda.end()));

  return std::nullopt;
}

}  // namespace chd
```

Listing 13: 03-divisao-e-conquista/main.cpp

```
#include "00-comum/execucao.hpp"
```

```cpp
#include "03-divisao-e-conquista/ciclo_por_blocos.hpp"

int main(int argc, char** argv) {
  return chd::executa(argc, argv, "ciclo_por_blocos",
                      [](const chd::Digrafo& g, const chd::Contexto&, chd::Contador&
    contador,
                         const chd::Relogio&) {
                        chd::Resultado resultado;
                        resultado.ciclo = chd::ciclo_por_blocos_contiguos(g, contador)
    ;
                        resultado.motivo = "nao_encontrou";
                        return resultado;
                      });
}
```

3.5 Programação dinâmica (Held-Karp)

Estado: $\mathrm{dp}[S][v]$ é verdadeiro quando existe caminho simples que começa em 0, visita exatamente o conjunto S e termina em v, com $0 \in S$ e $v \in S$. Base: $\mathrm{dp}[\{0\}][0] =$ verdadeiro. Transição: $\mathrm{dp}[S][v]$ é verdadeiro quando existe $u \in S \setminus \{v\}$ com $\mathrm{dp}[S \setminus \{v\}][u]$ verdadeiro e arco (u, v). Há ciclo quando existe $v \neq 0$ com $\mathrm{dp}[V][v]$ verdadeiro e arco $(v, 0)$ [9, 2].

Cada conjunto é uma máscara de bits, processada em ordem crescente (a máscara sem o bit de v é menor e já foi calculada). Para cada máscara guarda-se um conjunto de vértices finais possíveis (um inteiro de 32 bits) e, para cada estado, o predecessor, de modo que o ciclo é reconstruído, e não só o booleano. A função aceita até $n = 22$ e, acima disso, lança erro em vez de responder que o ciclo não existe.

Listing 14: `held_karp.cpp`

```cpp
#include "04-programacao-dinamica/held_karp.hpp"

#include <algorithm>
#include <cstdint>
#include <stdexcept>

namespace chd {

std::optional<std::vector<int>> held_karp(const Digrafo& g, Contador& contador,
                                          const Relogio& relogio) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }
  if (n > 22) {
    throw std::invalid_argument("Held-Karp␣deste␣trabalho␣cobre␣n␣<=␣22");
  }
  if (!g.fortemente_conexo()) {
    return std::nullopt;
  }

  const uint32_t limite = 1u << n;
  std::vector<uint32_t> alcanca(limite, 0);
  std::vector<int> predecessor(static_cast<size_t>(limite) * n, -1);

```

```
  // Base: o caminho que só contém a origem, terminando nela.
  alcanca[1u] = 1u;
  contador.escritas++;

  for (uint32_t mascara = 1; mascara < limite; ++mascara) {
    if ((mascara & 1023u) == 0) {
      relogio.verifica();
    }
    if ((mascara & 1u) == 0 || alcanca[mascara] == 0) {
      continue;
    }
    for (uint32_t bits = alcanca[mascara]; bits != 0; bits &= bits - 1) {
      const int v = __builtin_ctz(bits);
      contador.nos++;
      for (int u : g.saida(v)) {
        contador.consultas_aresta++;
        if ((mascara & (1u << u)) != 0) {
          continue;
        }
        const uint32_t nova = mascara | (1u << u);
        const uint32_t bit_u = 1u << u;
        if ((alcanca[nova] & bit_u) == 0) {
          alcanca[nova] |= bit_u;
          predecessor[static_cast<size_t>(nova) * n + u] = v;
          contador.escritas++;
        }
      }
    }
  }

  const uint32_t cheia = limite - 1;
  int fim = -1;
  for (int v = 1; v < n; ++v) {
    if ((alcanca[cheia] & (1u << v)) != 0 && g.consulta_arco(v, 0, contador)) {
      fim = v;
      break;
    }
  }
  if (fim < 0) {
    return std::nullopt;
  }

  std::vector<int> ciclo;
  ciclo.reserve(n);
  uint32_t mascara = cheia;
  int v = fim;
  while (true) {
    ciclo.push_back(v);
    if (v == 0) {
      break;
    }
    const int anterior = predecessor[static_cast<size_t>(mascara) * n + v];
    mascara ^= 1u << v;
    v = anterior;
  }
  std::reverse(ciclo.begin(), ciclo.end());
  return ciclo;
}
```

```
84
85 }  // namespace chd
```

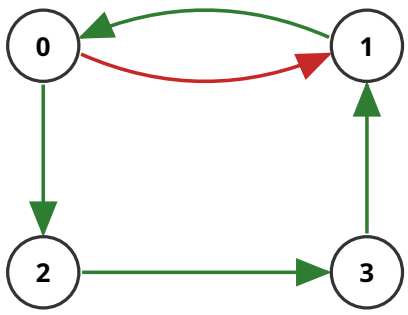
Listing 15: 04-programacao-dinamica/main.cpp

```
1  #include "00-comum/execucao.hpp"
2  #include "04-programacao-dinamica/held_karp.hpp"
3
4  int main(int argc, char** argv) {
5    return chd::executa(argc, argv, "held_karp",
6                        [](const chd::Digrafo& g, const chd::Contexto&, chd::Contador&
       contador,
7                           const chd::Relogio& relogio) {
8                          chd::Resultado resultado;
9                          resultado.ciclo = chd::held_karp(g, contador, relogio);
10                         resultado.motivo = "inexistente";
11                         return resultado;
12                       });
13 }
```

3.6 Guloso

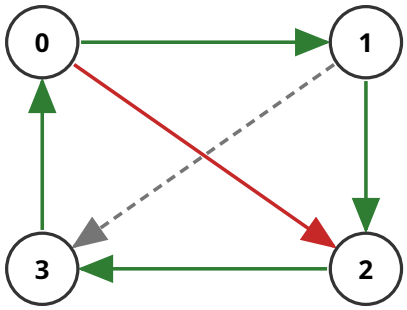
Dois critérios, mesma estrutura: parte de 0 e, enquanto houver vértice livre, escolhe um vizinho de saída livre; se empaca antes de cobrir V, ou se o último não aponta para 0, devolve falha. O critério `indice` escolhe o menor índice; o critério `warnsdorff` escolhe o vizinho com menos saídas livres. **Nenhum dos dois tem garantia**, e cada um tem contraexemplo automatizado:

Critério `indice`. Arcos $(0,1),(0,2),(2,3),(3,1),(1,0)$. O ciclo $0 \to 2 \to 3 \to 1 \to 0$ existe (verde). O guloso sai de 0 para 1 (vermelho), e de 1 só há arco para 0, que já está no caminho: falha num digrafo que tem ciclo.



Armadilha do critério por índice. Verde: ciclo existente. Vermelho: escolha do guloso.

Critério `warnsdorff`. Arcos $(0,1),(0,2),(1,2),(1,3),(2,3),(3,0)$. O ciclo $0 \to 1 \to 2 \to 3 \to 0$ existe (verde). Saindo de 0, o vértice 1 tem duas saídas livres e o 2 tem uma; a regra escolhe 2 (vermelho), segue a 3 e empaca com 1 de fora.



Armadilha do critério de Warnsdorff. Verde: ciclo existente. Vermelho: escolha do guloso. Tracejado: arco não usado.

Os dois são heurísticas gulosas, sem garantia de otimalidade e sem razão de aproximação.

Listing 16: `guloso.cpp`

```cpp
#include "05-guloso/guloso.hpp"

#include <algorithm>

namespace chd {
namespace {

struct Item {
  int vertice;
  int chave;
};

int saidas_livres(const Digrafo& g, int v, const std::vector<char>& usado, Contador&
    contador) {
  int livres = 0;
  for (int w : g.saida(v)) {
    contador.consultas_aresta++;
    if (!usado[w]) {
      livres++;
    }
  }
  return livres;
}

}  // namespace

std::optional<std::vector<int>> guloso(const Digrafo& g, Contador& contador,
    CriterioGuloso criterio) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }

  std::vector<int> caminho;
  caminho.reserve(n);
  caminho.push_back(0);
  std::vector<char> usado(n, 0);
  usado[0] = 1;
```

```
  contador.escritas++;

  while (static_cast<int>(caminho.size()) < n) {
    contador.nos++;
    const int u = caminho.back();
    std::vector<Item> candidatos;
    for (int v : g.saida(u)) {
      if (usado[v]) {
        continue;
      }
      int chave = v;
      if (criterio == CriterioGuloso::Warnsdorff) {
        chave = saidas_livres(g, v, usado, contador);
      }
      candidatos.push_back({v, chave});
    }
    if (candidatos.empty()) {
      return std::nullopt;
    }
    if (criterio == CriterioGuloso::Warnsdorff) {
      std::sort(candidatos.begin(), candidatos.end(), [](const Item& a, const Item& b)
     {
        if (a.chave != b.chave) {
          return a.chave < b.chave;
        }
        return a.vertice < b.vertice;
      });
    } else {
      std::sort(candidatos.begin(), candidatos.end(), [](const Item& a, const Item& b)
     {
        return a.vertice < b.vertice;
      });
    }
    const int escolhido = candidatos.front().vertice;
    usado[escolhido] = 1;
    caminho.push_back(escolhido);
    contador.escritas++;
  }

  if (!g.consulta_arco(caminho.back(), 0, contador)) {
    return std::nullopt;
  }
  return caminho;
}

}  // namespace chd
```

Listing 17: 05-guloso/main.cpp

```
#include "00-comum/execucao.hpp"
#include "05-guloso/guloso.hpp"

int main(int argc, char** argv) {
  return chd::executa(argc, argv, "guloso",
                      [](const chd::Digrafo& g, const chd::Contexto& contexto, chd::
    Contador& contador,
                         const chd::Relogio&) {
                       const bool indice = contexto.opcao("criterio", "indice") == "
```

```
    indice";
                        chd::Resultado resultado;
                        resultado.ciclo = chd::guloso(
                            g, contador, indice ? chd::CriterioGuloso::Indice : chd::
    CriterioGuloso::Warnsdorff);
                        resultado.motivo = "nao_encontrou";
                        return resultado;
                    });
}
```

3.7 Heurística: recozimento simulado

Reformulação de otimização usada pela heurística: entre as permutações com origem 0, maximizar o número de arcos presentes nas n posições do ciclo. O *fitness* vai de 0 a n, e o digrafo tem ciclo exatamente quando o máximo vale n. Essa reformulação permite busca local; **ela não cria razão de aproximação**.

Movimento: retirar um vértice e inseri-lo em outra posição (relocação). A *variação* do fitness depende só dos arcos da fronteira da posição antiga e da nova, e é calculada em $O(1)$. Aceita melhora sempre e aceita piora com probabilidade $e^{\Delta/T}$; esfriamento $T \leftarrow \alpha T$; vários recomeços a partir de permutações aleatórias; para ao achar fitness n ou ao gastar o orçamento [11].

Parâmetros declarados antes da bateria e congelados: $T_0 = 1$, $\alpha = 0{,}995$, $200 \cdot n$ movimentos por recomeço e 10 recomeços. O resultado separa `Ciclo` (ciclo válido encontrado) de `Orcamento` (orçamento esgotado). Esgotar o orçamento **não** prova que o ciclo não existe.

Listing 18: `recozimento.hpp`

```
#pragma once

#include "00-comum/digrafo.hpp"

#include <random>
#include <vector>

namespace chd {

// Parâmetros declarados antes da bateria. O piloto pode trocar alfa e,
// depois da escolha, o valor usado no relatório fica congelado.
struct ParamsRecozimento {
  double t0 = 1.0;
  double alfa = 0.995;
  int movimentos_por_n = 200;
  int recomecos = 10;
};

enum class MotivoHeuristica { Ciclo, Orcamento };

// ciclo_valido só é preenchido quando o fitness chega a n.
// motivo Orcamento com ciclo vazio não prova que o ciclo não existe.
struct ResultadoHeuristica {
  std::vector<int> ciclo_valido;
  std::vector<int> melhor_permutacao;
  int fitness = 0;
```

```
  MotivoHeuristica motivo = MotivoHeuristica::Orcamento;
};

int fitness_permutacao(const Digrafo& g, const std::vector<int>& perm, Contador&
    contador);

// Variação do fitness ao mover o vértice que está em `origem` para o índice `destino
    `.
// Só os arcos da fronteira mudam, então a conta é O(1).
int delta_relocacao(const Digrafo& g, const std::vector<int>& perm, int origem, int
    destino,
                    Contador& contador);

void aplica_relocacao(std::vector<int>& perm, int origem, int destino);

ResultadoHeuristica recozimento(const Digrafo& g, Contador& contador, std::mt19937&
    rng,
                                const ParamsRecozimento& params = {});

}  // namespace chd
```

Listing 19: `recozimento.cpp`

```
#include "06-heuristica/recozimento.hpp"

#include <algorithm>
#include <cmath>

namespace chd {

int fitness_permutacao(const Digrafo& g, const std::vector<int>& perm, Contador&
    contador) {
  const int n = static_cast<int>(perm.size());
  int fitness = 0;
  for (int i = 0; i < n; ++i) {
    const int proximo = perm[(i + 1) % n];
    if (g.consulta_arco(perm[i], proximo, contador)) {
      fitness++;
    }
  }
  return fitness;
}

void aplica_relocacao(std::vector<int>& perm, int origem, int destino) {
  if (origem == destino) {
    return;
  }
  const int vertice = perm[origem];
  if (origem < destino) {
    for (int k = origem; k < destino; ++k) {
      perm[k] = perm[k + 1];
    }
    perm[destino] = vertice;
  } else {
    for (int k = origem; k > destino; --k) {
      perm[k] = perm[k - 1];
    }
    perm[destino] = vertice;
```

```
35    }
36  }
37
38  int delta_relocacao(const Digrafo& g, const std::vector<int>& perm, int origem, int
        destino,
39                      Contador& contador) {
40    if (origem == destino) {
41      return 0;
42    }
43    const int n = static_cast<int>(perm.size());
44    auto presente = [&](int u, int v) {
45      return g.consulta_arco(u, v, contador) ? 1 : 0;
46    };
47    auto sucessor = [&](int i) { return perm[(i + 1) % n]; };
48
49    if (origem < destino) {
50      const int anterior = perm[origem - 1];
51      const int vertice = perm[origem];
52      const int primeiro = perm[origem + 1];
53      const int ultimo = perm[destino];
54      const int depois = sucessor(destino);
55      const int perdidos = presente(anterior, vertice) + presente(vertice, primeiro) +
        presente(ultimo, depois);
56      const int ganhos = presente(anterior, primeiro) + presente(ultimo, vertice) +
        presente(vertice, depois);
57      return ganhos - perdidos;
58    }
59
60    const int anterior = perm[destino - 1];
61    const int primeiro = perm[destino];
62    const int ultimo = perm[origem - 1];
63    const int vertice = perm[origem];
64    const int depois = sucessor(origem);
65    const int perdidos = presente(anterior, primeiro) + presente(ultimo, vertice) +
        presente(vertice, depois);
66    const int ganhos = presente(anterior, vertice) + presente(vertice, primeiro) +
        presente(ultimo, depois);
67    return ganhos - perdidos;
68  }
69
70  ResultadoHeuristica recozimento(const Digrafo& g, Contador& contador, std::mt19937&
        rng,
71                                  const ParamsRecozimento& params) {
72    ResultadoHeuristica resultado;
73    const int n = g.n();
74    if (n < 2) {
75      return resultado;
76    }
77
78    std::vector<int> perm(n);
79    perm[0] = 0;
80    for (int i = 1; i < n; ++i) {
81      perm[i] = i;
82    }
83
84    auto guarda_melhor = [&](const std::vector<int>& atual, int fitness) {
85      if (fitness > resultado.fitness || resultado.melhor_permutacao.empty()) {
86        resultado.fitness = fitness;
```

```
      resultado.melhor_permutacao = atual;
      contador.escritas++;
    }
  };

  std::uniform_int_distribution<int> indice(1, n - 1);
  std::uniform_real_distribution<double> unidade(0.0, 1.0);

  for (int recomeço = 0; recomeço < params.recomecos; ++recomeço) {
    std::shuffle(perm.begin() + 1, perm.end(), rng);
    int fitness = fitness_permutacao(g, perm, contador);
    guarda_melhor(perm, fitness);
    if (fitness == n) {
      resultado.ciclo_valido = perm;
      resultado.motivo = MotivoHeuristica::Ciclo;
      return resultado;
    }

    double temperatura = params.t0;
    const int movimentos = params.movimentos_por_n * n;
    for (int passo = 0; passo < movimentos; ++passo) {
      contador.nos++;
      int origem = indice(rng);
      int destino = indice(rng);
      if (destino == origem) {
        destino = origem == n - 1 ? 1 : origem + 1;
      }
      const int delta = delta_relocacao(g, perm, origem, destino, contador);
      const bool melhora = delta >= 0;
      const bool aceita =
          melhora || (temperatura > 0.0 && unidade(rng) < std::exp(static_cast<double
    >(delta) / temperatura));
      if (aceita) {
        aplica_relocacao(perm, origem, destino);
        fitness += delta;
        guarda_melhor(perm, fitness);
        if (fitness == n) {
          resultado.ciclo_valido = perm;
          resultado.motivo = MotivoHeuristica::Ciclo;
          return resultado;
        }
      }
      temperatura *= params.alfa;
    }
  }

  resultado.motivo = MotivoHeuristica::Orcamento;
  return resultado;
}

}  // namespace chd
```

Listing 20: 06-heuristica/main.cpp

```
#include "00-comum/execucao.hpp"
#include "06-heuristica/recozimento.hpp"

#include <random>
```

```cpp

int main(int argc, char** argv) {
  return chd::executa(argc, argv, "recozimento",
                      [](const chd::Digrafo& g, const chd::Contexto& contexto, chd::Contador& contador,
                         const chd::Relogio&) {
                        std::mt19937 rng(20261107u + static_cast<unsigned>(contexto.n) * 17u +
                                         static_cast<unsigned>(contexto.rep));
                        const chd::ResultadoHeuristica achado = chd::recozimento(g, contador, rng);
                        chd::Resultado resultado;
                        resultado.fitness = achado.fitness;
                        resultado.motivo = "orcamento";
                        if (achado.motivo == chd::MotivoHeuristica::Ciclo) {
                          resultado.ciclo = achado.ciclo_valido;
                        }
                        return resultado;
                      });
}
```

3.8 Caso especial: torneios (Camion)

Teorema 1 (Camion, 1959 [4]) *Um torneio com $n \geq 3$ vértices tem ciclo hamiltoniano direcionado se e somente se é fortemente conexo.*

Este módulo não é uma das seis estratégias do caso geral: mostra a fronteira da dificuldade, pois a mesma pergunta, restrita a torneios fortes, cai em P. Construção: (i) caminho hamiltoniano por inserção; (ii) um arco de volta ao início do caminho fecha um ciclo que cobre um prefixo; (iii) cada vértice de fora que encaixa entre dois consecutivos do ciclo entra ali; (iv) quando nenhum encaixa sozinho, os vértices de fora se dividem entre os que apontam para o ciclo inteiro e os que recebem arco de todo o ciclo, e um arco do segundo grupo para o primeiro permite inserir um par. O módulo recusa o digrafo que não é torneio.

Listing 21: `torneio.cpp`

```cpp
#include "07-torneio/torneio.hpp"

#include <algorithm>
#include <cstddef>
#include <stdexcept>

namespace chd {
namespace {

std::vector<int> caminho_hamiltoniano(const Digrafo& g, Contador& contador) {
  const int n = g.n();
  std::vector<int> caminho;
  caminho.reserve(n);
  caminho.push_back(0);
  for (int v = 1; v < n; ++v) {
    int posicao = static_cast<int>(caminho.size());
    for (int i = 0; i < static_cast<int>(caminho.size()); ++i) {
      if (g.consulta_arco(v, caminho[i], contador)) {
```

```
19          posicao = i;
20          break;
21        }
22      }
23      caminho.insert(caminho.begin() + posicao, v);
24      contador.escritas++;
25      contador.nos++;
26    }
27    return caminho;
28  }
29
30  }  // namespace
31
32  std::optional<std::vector<int>> ciclo_em_torneio(const Digrafo& g, Contador& contador)
        {
33    const int n = g.n();
34    if (!g.eh_torneio() || n < 3 || !g.fortemente_conexo()) {
35      return std::nullopt;
36    }
37
38    const std::vector<int> caminho = caminho_hamiltoniano(g, contador);
39    int fecho = -1;
40    for (int i = n - 1; i >= 2; --i) {
41      if (g.consulta_arco(caminho[i], caminho[0], contador)) {
42        fecho = i;
43        break;
44      }
45    }
46    if (fecho < 0) {
47      throw std::logic_error("torneio forte sem aresta de volta ao inicio do caminho");
48    }
49
50    std::vector<int> ciclo(caminho.begin(), caminho.begin() + fecho + 1);
51    std::vector<char> dentro(n, 0);
52    for (int v : ciclo) {
53      dentro[v] = 1;
54    }
55    std::vector<int> fora;
56    for (int v = 0; v < n; ++v) {
57      if (!dentro[v]) {
58        fora.push_back(v);
59      }
60    }
61
62    while (!fora.empty()) {
63      bool inseriu = false;
64      for (size_t indice = 0; indice < fora.size() && !inseriu; ++indice) {
65        const int v = fora[indice];
66        const int m = static_cast<int>(ciclo.size());
67        for (int i = 0; i < m; ++i) {
68          const int atual = ciclo[i];
69          const int proximo = ciclo[(i + 1) % m];
70          if (g.consulta_arco(atual, v, contador) && g.consulta_arco(v, proximo,
      contador)) {
71            ciclo.insert(ciclo.begin() + i + 1, v);
72            fora.erase(fora.begin() + static_cast<std::ptrdiff_t>(indice));
73            contador.escritas++;
74            contador.nos++;
```

```
75            inseriu = true;
76            break;
77          }
78        }
79      }
80      if (inseriu) {
81        continue;
82      }
83
84      std::vector<int> domina;
85      std::vector<int> dominado;
86      for (int v : fora) {
87        bool vai_a_todos = true;
88        bool recebe_de_todos = true;
89        for (int c : ciclo) {
90          if (!g.tem_arco(v, c)) {
91            vai_a_todos = false;
92          }
93          if (!g.tem_arco(c, v)) {
94            recebe_de_todos = false;
95          }
96        }
97        if (vai_a_todos) {
98          domina.push_back(v);
99        } else if (recebe_de_todos) {
100         dominado.push_back(v);
101       } else {
102         throw std::logic_error("vertice misto sem posicao de insercao no ciclo");
103       }
104     }
105     if (domina.empty() || dominado.empty()) {
106       throw std::logic_error("particao de Camion vazia num torneio forte");
107     }
108
109     int origem = -1;
110     int destino = -1;
111     for (int b : dominado) {
112       for (int a : domina) {
113         contador.consultas_aresta++;
114         if (g.tem_arco(b, a)) {
115           origem = b;
116           destino = a;
117           break;
118         }
119       }
120       if (origem >= 0) {
121         break;
122       }
123     }
124     if (origem < 0) {
125       throw std::logic_error("torneio forte sem arco do conjunto dominado para o
      dominante");
126     }
127
128     ciclo.insert(ciclo.begin() + 1, destino);
129     ciclo.insert(ciclo.begin() + 1, origem);
130     contador.escritas += 2;
131     contador.nos++;
```

```
    fora.erase(std::remove_if(fora.begin(), fora.end(),
                              [&](int v) { return v == origem || v == destino; }),
               fora.end());
  }

  return ciclo;
}

}  // namespace chd
```

Listing 22: 07-torneio/main.cpp

```
#include "00-comum/execucao.hpp"
#include "07-torneio/torneio.hpp"

int main(int argc, char** argv) {
  return chd::executa(argc, argv, "camion",
                      [](const chd::Digrafo& g, const chd::Contexto&, chd::Contador&
    contador,
                         const chd::Relogio&) {
                        chd::Resultado resultado;
                        if (!g.eh_torneio()) {
                          resultado.motivo = "recusado";
                          return resultado;
                        }
                        resultado.ciclo = chd::ciclo_em_torneio(g, contador);
                        resultado.motivo = "inexistente";
                        return resultado;
                      });
}
```

4 Análise de complexidade

4.1 Tempo e espaço teóricos

Usa-se $n = |V|$ e matriz de adjacência, com consulta de arco em $O(1)$.

Função	Tempo	Espaço extra	Garantia
`verifica_ciclo`	$O(n)$	$O(n)$	exata
Força bruta	$\Theta(n \cdot (n-1)!)$ no pior caso	$O(n)$	exata
Backtracking	$O(n!)$ no pior caso; depende da densidade	$O(n)$ (pilha)	exata
Held-Karp	$\Theta(n^2\, 2^n)$	$\Theta(n\, 2^n)$	exata
Guloso, índice	$O(n^2 \log n)$	$O(n)$	nenhuma
Guloso, Warnsdorff	$O(n^3)$	$O(n)$	nenhuma
Recozimento	$O(r \cdot c\, n^2)$, r recomeços, $c = 200$	$O(n)$	nenhuma
Camion (só torneio)	$O(n^3)$	$O(n)$	exata
Blocos contíguos	$O(n! \cdot n)$ $(n \leq 10)$	$O(n)$	incorreto

Força bruta. Gera $(n-1)!$ permutações; cada uma consulta até n arcos e copia n posições. O melhor caso termina na primeira permutação; o pior caso é $\Theta(n \cdot (n-1)!)$, e é o caso dos digrafos sem ciclo. Espaço $O(n)$, pois `next_permutation` gera no lugar.

Backtracking. A árvore tem profundidade n e grau no máximo $n-1$, portanto $O(n!)$ nós no pior caso. Com a poda de vértice já usado, o número de nós em um digrafo completo sobre m vértices é $\sum_k m!/(m-k)! \approx e \cdot m!$. Para o espaço, a pilha de recursão e o vetor de usados são $O(n)$.

Held-Karp. São 2^n máscaras, para cada uma até n vértices finais e para cada vértice até n vizinhos de saída: $\Theta(n^2\, 2^n)$. O tempo depende de n e não do número de arcos, exceto pelas podas por máscara inalcançável. O espaço é dominado pela tabela de predecessores, com $n\, 2^n$ inteiros de 4 bytes, mais 2^n palavras de 32 bits do conjunto de finais. Para $n = 22$ são $22 \cdot 2^{22} \cdot 4 \approx 369$ MB mais 16 MB, aproximadamente 385 MB. Por isso a memória limita o n *antes* do tempo, e o código recusa $n > 22$.

Guloso. Cada passo percorre os vizinhos de saída ($O(n)$) e ordena os candidatos ($O(n \log n)$); são $n-1$ passos, daí $O(n^2 \log n)$. No critério de Warnsdorff, cada candidato conta suas próprias saídas livres ($O(n)$), o que leva a $O(n^3)$. (Trocar a ordenação por uma busca do mínimo reduziria para $O(n^2)$ e $O(n^3)$ sem mudar a resposta; não foi feito porque o objetivo do trabalho é a comparação.)

Recozimento. O cálculo da variação é $O(1)$ (seis consultas de arco), mas aplicar a relocação aceita desloca até n posições, então um movimento custa $O(n)$ quando aceito. Com r recomeços de $c\,n$ movimentos, o tempo é $O(r\, c\, n^2)$, proporcional ao orçamento declarado. Espaço $O(n)$.

Camion. O caminho por inserção custa $O(n^2)$ (cada vértice varre o caminho). Cada rodada de inserção percorre os vértices de fora e o ciclo, $O(n^2)$, e pode haver até n rodadas, portanto $O(n^3)$ no pior caso. Para $n \leq 30$ no experimento o tempo é irrelevante frente aos métodos exponenciais.

4.2 O que o contador mede

A contagem de instruções usa as três grandezas do `Contador`. A *unidade comum* de comparação entre estratégias é o tempo de parede. Cada algoritmo conta "nós" de uma natureza distinta, e por isso o texto não soma contagens de algoritmos diferentes num "total de instruções" único.

Algoritmo	O que é um “nó”
Força bruta	uma permutação testada (valor medido: exatamente $(n-1)!$ no pior caso)
Backtracking	uma chamada recursiva da busca
Held-Karp	um estado (máscara, vértice final) expandido
Guloso	uma escolha de vizinho
Recozimento	um movimento de relocação proposto
Camion	uma inserção

5 Análise de resultados

5.1 Método

Digrafos de teste. Semente base 20261107. Seis famílias de digrafos, cada uma com gerador próprio:

- **Ciclo + arcos extras**: um ciclo plantado mais n arcos aleatórios (resposta sim).
- **DAG aleatório**: arcos só para a frente, $p = 0{,}5$ (resposta não; não é fortemente conexo).
- **Erdős–Rényi direcionado**, com $p = 0{,}35$ fixo e com $p = (\ln n)/n$, a ordem do limiar de aparecimento de grau mínimo de entrada e saída 1 [7]. Uma amostra finita não é tratada como demonstração do teorema.
- **Torneio aleatório**: sim quando é forte.
- **Pior caso (sem ciclo)**: um digrafo completo sobre $n-1$ vértices mais um vértice x ligado apenas ao vértice 1 (arcos $1 \to x$ e $x \to 1$). O digrafo é fortemente conexo, não tem ciclo hamiltoniano (passar por x exige usar 1 duas vezes) e obriga os métodos exatos a esgotar a busca.

Juiz. Até $n = 14$ o Held-Karp julga os demais; até $n = 11$ a força bruta concorda com ele (nos 288 digrafos em que mais de um método exato rodou, as respostas coincidiram). Todo ciclo devolvido passa por `verifica_ciclo`. Acima do alcance da PD, “ciclo válido encontrado” ou “orçamento esgotado”; a palavra “inexistente” fica reservada aos métodos exatos.

Execução. AMD Ryzen 7 5700G, 31 GiB de memória, g++ 16.2 com `-O2`, um núcleo. Foram 2528 medições, com 8 digrafos por tamanho e família. Timeout de 30 s por chamada exata: 0 medições foram censuradas. Tamanhos: força bruta e backtracking $n = 6$ a 11; Held-Karp $n = 6$ a 14; guloso, recozimento e Camion até $n = 30$. Os números de nós são reproduzíveis com os programas do trabalho, pois a semente é fixa; os tempos variam um pouco de uma execução para outra.

Cuidado com a resolução do relógio. Tempos abaixo de aproximadamente 0,005 ms são ruído de medição (resolução de 0,001 ms nos dados) e não são interpretados. A discussão se apoia em tempos acima de 0,01 ms e nas contagens de nós, que não têm ruído.

5.2 Os três exatos no pior caso

	Força bruta		Backtracking		Held-Karp	
n	nós	ms	nós	ms	nós	ms
6	120	0.002	81	0.002	41	0.001
7	720	0.009	391	0.010	97	0.001
8	5040	0.075	2283	0.053	225	0.003
9	40320	0.751	15657	0.380	513	0.005
10	362880	6.029	123301	3.049	1153	0.026
11	3628800	57.604	1096011	38.012	2561	0.070
12	–	–	–	–	5633	0.150
13	–	–	–	–	12289	0.385
14	–	–	–	–	26625	0.955

A Figura 1 e a Figura 2 mostram os mesmos dados em escala logarítmica.

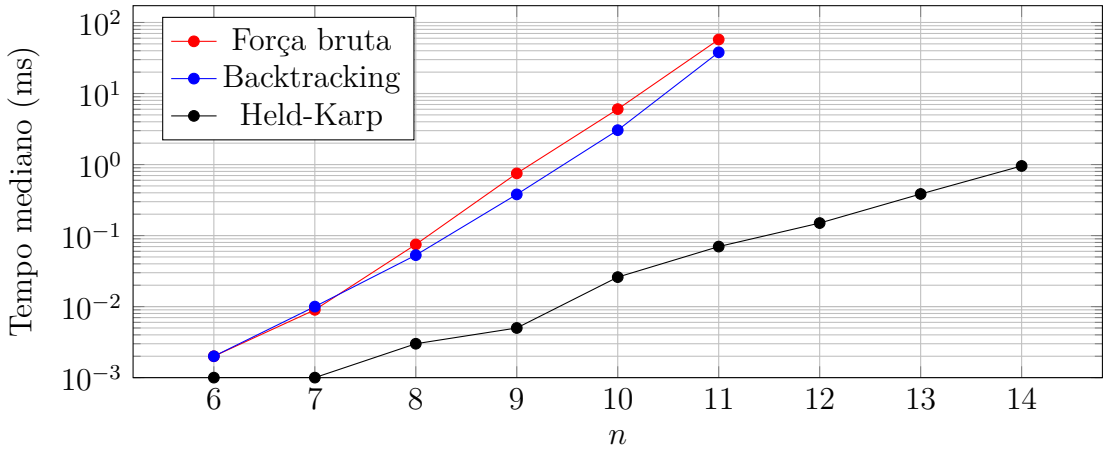


Figura 1: Tempo mediano contra n na família pior caso (escala logarítmica).

Força bruta. Os nós medidos são exatamente $(n-1)!$: 120 em $n = 6$ e 3 628 800 em $n = 11$, igual à teoria. O tempo cresce por um fator de cerca de 9,6 entre $n = 10$ e $n = 11$ (6,03 → 57,6 ms), compatível com o fator $n - 1 = 10$.

Backtracking. A poda de vértice usado reduz os nós a cerca de 30% da força bruta em $n = 11$ (1 096 011 contra 3 628 800), valor próximo de $e \cdot 9! \approx 986\,000$ da fórmula da seção anterior. O ganho é de uma constante, não da ordem: a razão entre nós consecutivos continua a crescer com n. Nesta família as duas ordens de vizinhos têm o mesmo número de nós (esperado: o grafo é completo sobre $n - 1$ vértices e o digrafo não tem ciclo, logo a árvore é esgotada de qualquer modo); a ordem de Warnsdorff custa mais tempo pelo cálculo da chave (43,6 contra 38,0 ms).

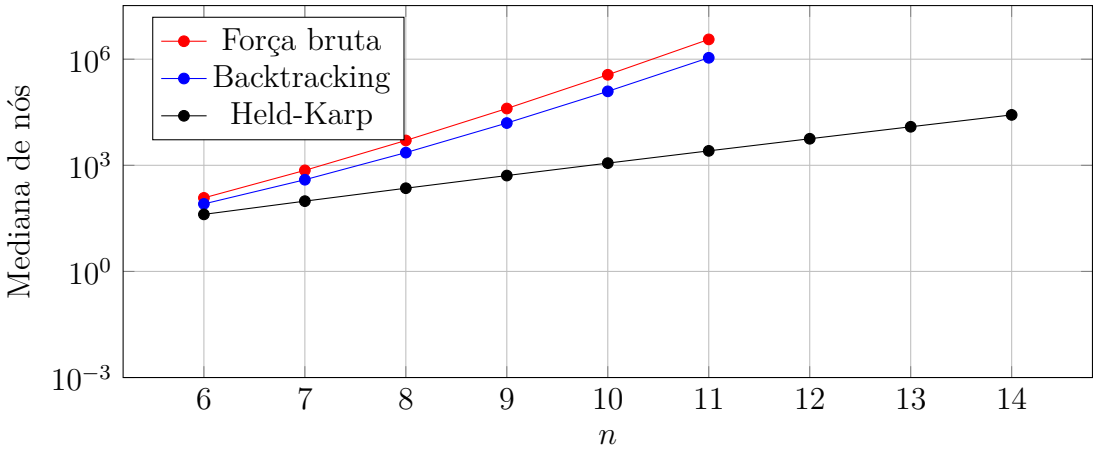


Figura 2: Mediana de nós contra n na família pior caso (escala logarítmica).

Held-Karp. Os nós crescem por um fator de aproximadamente 2,2 por unidade de n ($2561 \to 5633 \to 12289 \to 26625$), compatível com $\Theta(n^2\,2^n)$, e o tempo cresce por cerca de 2,5 por unidade de n (0,150 → 0,385 → 0,955 ms). Em $n = 11$, Held-Karp leva 0,070 ms contra 57,6 ms da força bruta, uma razão de cerca de 800 vezes, e a razão aumenta com n. O cruzamento entre o fatorial e $n^2\,2^n$ acontece já em $n \approx 7$, onde as duas curvas são indistinguíveis dentro do ruído; a partir daí a diferença é de ordem.

5.3 Efeito do filtro de conectividade forte

Na família DAG a resposta é sempre não. Com o filtro, o backtracking responde sem expandir nenhum nó; sem o filtro, expande alguns:

n	nós com filtro	nós sem filtro
6	0	6
7	0	16
8	0	14
9	0	15
10	0	44
11	0	31

O filtro corta a árvore inteira, mas é *barato e correto*, e não resolve o caso forte: na família pior caso, que passa no filtro, a busca esgota 1,1 milhão de nós em $n = 11$.

5.4 Variação entre famílias

Família ($n = 11$)	Força bruta mediana	Força bruta máximo	Backtracking (Warnsdorff) mediana	Backtracking (Warnsdorff) máximo	Held-Karp mediana	Held-Karp máximo
Ciclo + arcos extras	0.000	0.000	0.001	0.002	0.004	0.049
DAG aleatório	0.000	0.000	0.000	0.000	0.000	0.001
Erdős–Rényi, $p = 0{,}35$	3.131	36.111	0.005	0.030	0.016	0.044
Erdős–Rényi, $p = \ln n/n$	0.000	0.000	0.000	0.000	0.001	0.001
Torneio aleatório	0.822	14.243	0.002	0.039	0.058	0.076

Tempos em milissegundos, $n = 11$, oito digrafos por família.

Nas famílias em que existe ciclo e há muitos arcos, o backtracking acha rapidamente (cauda curta). A força bruta tem cauda longa nas famílias com ciclo: em Erdős–Rényi com $p = 0{,}35$ a mediana é de 3,1 ms e o máximo, 36 ms, pois a primeira permutação que fecha pode estar tarde na ordem lexicográfica. O Held-Karp é estável (o tempo muda pouco entre famílias do mesmo n nas que existe ciclo), como esperado de uma complexidade que não depende da densidade. Mediana e máximo contam histórias diferentes, e os dois são publicados.

5.5 Guloso e recozimento: quem acha o ciclo?

Família	Digrafos com ciclo	Guloso índice	Guloso Warnsdorff	Recozimento
Ciclo + arcos extras	72	48/48	10/48	47/48
DAG aleatório	0	–	–	–
Erdős–Rényi, $p = 0{,}35$	41	1/27	0/27	27/27
Erdős–Rényi, $p = \ln n/n$	3	0/3	0/3	3/3
Torneio aleatório	62	6/42	3/42	42/42
Pior caso (sem ciclo)	0	–	–	–

Digrafos em que o Held-Karp (juiz) confirma ciclo, $n = 6$ a 14. A segunda coluna é o total de digrafos com ciclo; as heurísticas rodam só em parte dos tamanhos, então cada célula é encontrados / digrafos com ciclo *em que a heurística rodou*.

- **O guloso por índice acerta 48 de 48 na família ciclo + arcos extras por um efeito do gerador**: o ciclo plantado é $0 \to 1 \to 2 \to \cdots$, que é exatamente a ordem de índice. Isso mede o viés da família, e não a qualidade do critério; nas demais famílias (Erdős–Rényi, torneio) o mesmo critério encontra 1/27 e 6/42.
- **A regra de Warnsdorff** é pior que o índice nesses digrafos (10/48, 0/27, 3/42). O contraexemplo da Seção 3.6 mostra que é falha de projeto, e não azar.
- **O recozimento** encontra 47/48, 27/27, 3/3 e 42/42 dos ciclos, recuperando a maior parte das falhas dos gulosos, no mesmo orçamento fixo. Mas continua sem como provar que o ciclo não existe: nos digrafos sem ciclo (DAG, pior caso) ele sempre termina por `Orcamento`, e isso não prova nada.
- A família de limiar $p = (\ln n)/n$ tem só 3 digrafos com ciclo no total. A amostra é pequena e não sustenta conclusão estatística sobre o limiar.

5.6 Camion em torneios

Em torneios fortes, o algoritmo de Camion devolve o ciclo em tempo desprezível (0,003 ms em $n = 30$, 29 nós, um por vértice) enquanto o Held-Karp em $n = 14$ já gasta cerca de 0,9 ms e 34 827 nós. O mesmo problema, restrito a torneios, é polinomial; isso é o que a seção de aplicações chamou de fronteira da dificuldade.

6 Conclusão

O trabalho implementa, com contagem de instruções e medição de tempo, cinco das seis estratégias pedidas (força bruta, backtracking, programação dinâmica, guloso e recozimento como heurística) e justifica por escrito por que divisão e conquista não se sustenta no caso geral. O resultado experimental principal é a separação por ordem de grandeza: força bruta cresce com $(n-1)!$ nós medidos, backtracking diminui a constante e continua fatorial, e Held-Karp cresce com 2^n, com razão de cerca de 800 vezes em $n = 11$.

Limites. O teto prático do Held-Karp é a memória, não o tempo: a tabela de predecessores chega a cerca de 385 MB em $n = 22$. O guloso não tem garantia e erra nos contraexemplos da Seção 3.6. O recozimento recupera quase todos os ciclos nos testes, mas nunca prova inexistência, e por isso nunca decreta "não existe": ele separa "ciclo válido" de "orçamento esgotado". Os tempos abaixo de 0,005 ms foram descartados por resolução do relógio, e a família de limiar teve poucos digrafos com ciclo.

Dificuldades. (i) A reconstrução do ciclo no Held-Karp exige guardar o predecessor de cada estado, o que multiplica a memória por n. (ii) A tentação de apresentar o particionamento em blocos como divisão e conquista; o contraexemplo mostrou o falso negativo, e o trabalho o expõe em vez de o esconder. (iii) A família aleatória ingênua dá digrafos ou triviais (DAG) ou fáceis (arcos extras), e a comparação entre os exatos só ficou informativa com a família de pior caso. (iv) Distinguir, em todos os métodos, "não encontrou" de "não existe", para não afirmar o que a heurística não prova.

Referências

[1] L. M. Adleman. Molecular computation of solutions to combinatorial problems. *Science*, 266(5187):1021–1024, 1994.

[2] R. Bellman. Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM*, 9(1):61–63, 1962.

[3] A. Björklund. Determinant sums for undirected Hamiltonicity. In *Proc. 51st IEEE FOCS*, 2010.

[4] P. Camion. Chemins et circuits hamiltoniens des graphes complets. *Comptes Rendus de l'Académie des Sciences*, 249:2151–2152, 1959.

[5] P. E. C. Compeau, P. A. Pevzner, G. Tesler. How to apply de Bruijn graphs to genome assembly. *Nature Biotechnology*, 29:987–991, 2011.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*. 3. ed., MIT Press/McGraw-Hill, 2009.

[7] A. M. Frieze. An algorithm for finding Hamilton cycles in random directed graphs. *Journal of Algorithms*, 9(2):181–204, 1988.

[8] M. R. Garey, D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[9] M. Held, R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the SIAM*, 10(1):196–210, 1962.

[10] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, Plenum, 1972, pp. 85–103.

[11] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[12] N. Ziviani. *Projeto de Algoritmos com implementações em Java e C++*. Cengage Learning (Thomson/Pioneira), São Paulo, 1. ed., 2006.